

**LAPORAN TUGAS 6 SISTEM OPERASI
DEADLOCK**



Dosen Pembimbing:
Triawan Adi Cahyanto M.Kom

Dibuat Oleh:
Nadianur Anisa (2410651026)

TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER

2025

TUGAS 1 ANALISIS DEADLOCK

Jalankan program `deadlock_simple.c` dan jelaskan:

1. Mengapa program mengalami deadlock?

Program mengalami deadlock karena kedua thread saling menunggu resource yang sedang dipegang thread lain.

Contohnya: Thread 1 mengunci Lock 1, lalu ingin mengambil Lock 2 (yang sedang dipegang Thread 2). Thread 2 mengunci Lock 2, lalu ingin mengambil Lock 1 (yang sedang dipegang Thread 1). Jadi keduanya akan menunggu satu sama lain tanpa pernah melepas lock sehingga program akan berhenti total.

2. Kondisi deadlock apa saja yang terpenuhi?

- Mutual Exclusion
Setiap lock hanya bisa dipakai 1 thread dalam 1 waktu
- Hold and Wait
Thread 1 memegang lock 1 sambil menunggu lock 2 dan thread 2 melakukan sebaliknya. No Preemption
- Lock tidak bisa direbut paksa jadi hanya bisa dilepas oleh thread yang memasangnya.
- Circular Wait
Thread 1 menunggu resource milik thread 2, dan thread 2 menunggu resource milik thread 1, sehingga berbentuk lingkaran.

3. Gambar diagram resource allocation graph-nya

Thread 1 → Lock 1 → Thread 2 → Lock 2 → Thread 1 (Terus Berulang) atau

dalam bentuk hubungan:

- T1 memegang Lock1
- T1 menunggu Lock2
- T2 memegang Lock2
- T2 menunggu Lock1

Ini membentuk cycle, yang merupakan indikator deadlock.

TUGAS 2 IMPLEMENTASI PENCEGAHAN

Modifikasi program deadlock_simple.c menggunakan teknik:

1. Lock ordering
2. Try-lock dengan retry
3. Bandingkan kedua metode tersebut

1. Lock Ordering

```
GNU nano 7.2 prevention_ordering.c
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;

// Fungsi ambil lock sesuai urutan (lock1 + lock2)
void acquire_locks_ordered() {
    pthread_mutex_lock(&lock1);
    pthread_mutex_lock(&lock2);
}

void release_locks() {
    pthread_mutex_unlock(&lock2);
    pthread_mutex_unlock(&lock1);
}

void* thread1_func(void* arg) {
    printf("Thread 1 mulai...\n");
    acquire_locks_ordered();
    printf("Thread 1 dapat kedua lock\n");
    sleep(1);
    release_locks();
    printf("Thread 1 selesai\n");
    return NULL;
}

void* thread2_func(void* arg) {
    printf("Thread 2 mulai...\n");
    acquire_locks_ordered();
    printf("Thread 2 dapat kedua lock\n");
    sleep(1);
    release_locks();
    printf("Thread 2 selesai\n");
    return NULL;
}

int main() {
    pthread_t t1, t2;

    printf("=== LOCK ORDERING ===\n\n");

    pthread_create(&t1, NULL, thread1_func, NULL);
    pthread_create(&t2, NULL, thread2_func, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("\nProgram selesai TANPA deadlock\n");
    return 0;
}
```

Outputnya:

```
dea@LAPTOP-H54VQ765:~/Praktikum_Deadlock$ gcc prevention_ordering.c -o ordering -lpthread
dea@LAPTOP-H54VQ765:~/Praktikum_Deadlock$ ./ordering
=== LOCK ORDERING ===

Thread 1 mulai...
Thread 1 dapat kedua lock
Thread 2 mulai...
Thread 1 selesai
Thread 2 dapat kedua lock
Thread 2 selesai

Program selesai TANPA deadlock
dea@LAPTOP-H54VQ765:~/Praktikum_Deadlock$ gcc prevention_trylock.c -o trylock -lpthread
```

2. Try Lock dengan Retry

```
GNU nano 7.2 prevention_trylock.c
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;

void* thread1_func(void* arg) {
    int try_count = 0;
    while (1) {
        pthread_mutex_lock(&lock1);
        printf("Thread 1 ambil Lock 1\n");

        if (pthread_mutex_trylock(&lock2) == 0) {
            printf("Thread 1 berhasil ambil kedua lock\n");
            sleep(1);
            pthread_mutex_unlock(&lock2);
            pthread_mutex_unlock(&lock1);
            break;
        } else {
            printf("Thread 1 gagal ambil Lock 2 → retry\n");
            pthread_mutex_unlock(&lock1);
            try_count++;
            usleep(100000); // 0.1 detik
        }
    }

    printf("Thread 1 selesai (retry: %d)\n", try_count);
    return NULL;
}

void* thread2_func(void* arg) {
    int try_count = 0;
    while (1) {
        pthread_mutex_lock(&lock2);
        printf("Thread 2 ambil Lock 2\n");

        if (pthread_mutex_trylock(&lock1) == 0) {
            printf("Thread 2 berhasil ambil kedua lock\n");
            sleep(1);
            pthread_mutex_unlock(&lock1);
            pthread_mutex_unlock(&lock2);
            break;
        } else {
            printf("Thread 2 gagal ambil Lock 1 → retry\n");
            pthread_mutex_unlock(&lock2);
            try_count++;
            usleep(150000); // 0.15 detik
        }
    }

    printf("Thread 2 selesai (retry: %d)\n", try_count);
    return NULL;
}

int main() {
    pthread_t t1, t2;

    printf("=== TRY-LOCK + RETRY ===\n\n");

    pthread_create(&t1, NULL, thread1_func, NULL);
    pthread_create(&t2, NULL, thread2_func, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("\nProgram selesai TANPA deadlock\n");
    return 0;
}
```

Output:

```
dea@LAPTOP-H54VQ765:~/Praktikum_Deadlock$ gcc prevention_trylock.c -o trylock -lpthread
dea@LAPTOP-H54VQ765:~/Praktikum_Deadlock$ ./trylock
=== TRY-LOCK + RETRY ===

Thread 1 ambil Lock 1
Thread 1 berhasil ambil kedua lock
Thread 1 selesai (retry: 0)
Thread 2 ambil Lock 2
Thread 2 berhasil ambil kedua lock
Thread 2 selesai (retry: 0)

Program selesai TANPA deadlock
dea@LAPTOP-H54VQ765:~/Praktikum_Deadlock$ |
```

3. Perbandingan

Lock Ordering

Kelebihannya adalah efisien, simple, dan tidak ada retry, sedangkan kekurangannya adalah butuh desain urutan lock yang konsisten di seluruh program.

Try_Lock dan Retry

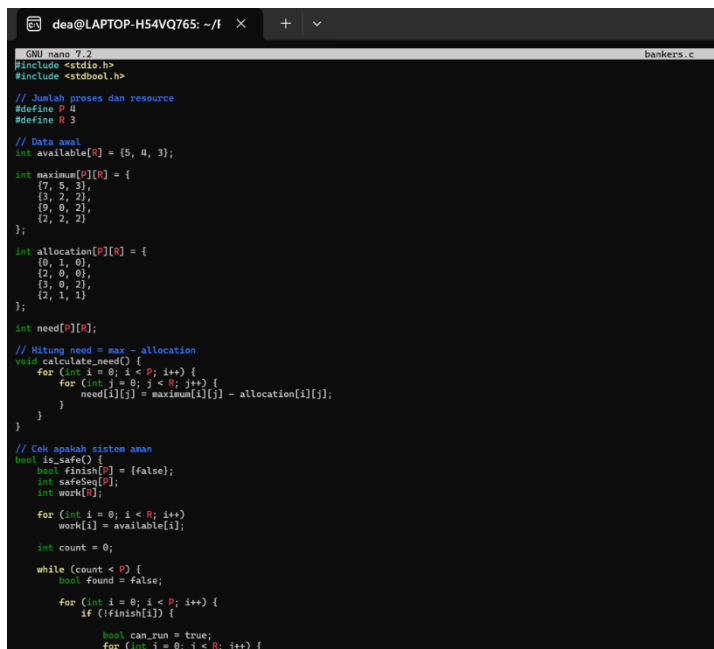
Kelebihannya adalah fleksibel, tidak harus sama dengan urutan lock nya, sedangkan kekurangannya adalah bisa banyak loop retry jadi tidak efisien.

TUGAS 3 BANKER'S ALGORITHM

Buat Program baru dengan data:

- 4 Proses
- 3 jenis resource
- Available: [5, 4, 3]
- Implementasikan request resource dan cek keamanan sistem

Buat file bankers.c



```
GNU nano 7.2 bankers.c
#include <stdio.h>
#include <stdbool.h>

// Jumlah proses dan resource
#define P 4
#define R 3

// Data awal
int available[R] = {5, 4, 3};

int maximum[P][R] = {
    {7, 5, 3},
    {3, 2, 2},
    {9, 0, 2},
    {2, 2, 2}
};

int allocation[P][R] = {
    {0, 1, 0},
    {2, 0, 0},
    {3, 0, 2},
    {2, 1, 1}
};

int need[P][R];

// Hitung need = max - allocation
void calculate_need() {
    for (int i = 0; i < P; i++) {
        for (int j = 0; j < R; j++) {
            need[i][j] = maximum[i][j] - allocation[i][j];
        }
    }
}

// Cek apakah sistem aman
bool is_safe() {
    bool finish[P] = {false};
    int safeSeq[P];
    int work[R];

    for (int i = 0; i < P; i++)
        work[i] = available[i];

    int count = 0;
    while (count < P) {
        bool found = false;
        for (int i = 0; i < P; i++) {
            if (!finish[i]) {
                bool can_run = true;
                for (int j = 0; j < R; j++) {
```

```

        if (need[i][j] > work[j]) {
            can_run = false;
            break;
        }

        if (can_run) {
            for (int j = 0; j < R; j++)
                work[j] += allocation[i][j];

            safeSeq[count++] = i;
            finish[i] = true;
            found = true;
        }
    }

    if (!found) {
        printf("\n[WARNING] Sistem tidak dalam safe state!\n");
        return false;
    }

    printf("\n[SUCCESS] Sistem dalam SAFE STATE.\nSafe sequence: ");
    for (int i = 0; i < P; i++)
        printf("P%d ", safeSeq[i]);

    printf("\n");
    return true;
}

// Fungsi request resource dari suatu proses
bool request_resources(int process, int request[]) {
    printf("\n== REQUEST dari P%d ==\n", process);
    printf("Meminta: ");
    for (int i = 0; i < R; i++) printf("%d ", request[i]);
    printf("\n");

    for (int i = 0; i < R; i++) {
        if (request[i] > need[process][i]) {
            printf("[DITOLAK] Request melebihi need!\n");
            return false;
        }
    }

    for (int i = 0; i < R; i++) {
        if (request[i] > available[i]) {
            printf("[DITOLAK] Resource tidak cukup!\n");
            return false;
        }
    }

    for (int i = 0; i < R; i++) {

```

```

        available[i] -= request[i];
        allocation[process][i] += request[i];
        need[process][i] -= request[i];
    }

    if (is_safe()) {
        printf("[DITERIMA] Request anan, dialokasikan.\n");
        return true;
    } else {
        printf("[DITOLAK] Request membuat sistem unsafe! Rollback.\n");

        for (int i = 0; i < R; i++) {
            available[i] += request[i];
            allocation[process][i] -= request[i];
            need[process][i] += request[i];
        }

        return false;
    }
}

void print_status() {
    printf("\n== STATUS SISTEM ==\n");

    printf("Available: ");
    for (int i = 0; i < R; i++) printf("%d ", available[i]);

    printf("\nAllocation:\n");
    for (int i = 0; i < P; i++) {
        printf("P%d: ", i);
        for (int j = 0; j < R; j++) printf("%d ", allocation[i][j]);
        printf("\n");
    }

    printf("\nNeed:\n");
    for (int i = 0; i < P; i++) {
        printf("P%d: ", i);
        for (int j = 0; j < R; j++) printf("%d ", need[i][j]);
        printf("\n");
    }
}

int main() {
    printf("== BANKER'S ALGORITHM - TUGAS 3 ==\n");

    calculate_need();
    print_status();

    // Cek keadaan awal
    is_safe();
}

```

```

// Request contoh (bebas kamu ganti)
int req1[R] = {1, 0, 2};
request_resources(1, req1);

print_status();

int req2[R] = {3, 3, 0};
request_resources(3, req2);

printf("\n== PROGRAM SELESAI ==\n");
return 0;
}

```

Penjelasan:

- $need[i][j] = max[i][j] - allocation[i][j]$; ini untuk menghitung kebutuhan sisa tiap proses.
- $work[j] = available[j]$; ini untuk menyalin resource yang tersedia untuk simulasi pengecekan keamanan.

- if (need[i][j] > work[j]) ini untuk mengecek apakah proses bisa dijalankan dengan resource yang ada
- work[j] += allocation[i][j];
ini jika resource telah selesai maka resource tersebut akan dikembalikan ke sistem.
- Pretend Allocation (alokasi sementara request)
Sistem seolah-olah memberikan resource untuk mengecek apakah aman atau tidak.
- Rollback jika unsafe
Jika sistem tidak aman, semua perubahan yang dilakukan akan dibatalakan untuk mencegah terjadinya deadlock
- if (isSafe()) ini adalah request yang hanya terjadi jika sistem tetap aman.

Outputnya:

```
dea@LAPTOP-H54VQ76S:~/Praktikum_Deadlock$ gcc bankers.c -o bankers
dea@LAPTOP-H54VQ76S:~/Praktikum_Deadlock$ ./bankers
=== BANKER'S ALGORITHM - TUGAS 3 ===

=== STATUS SISTEM ===
Available: 5 4 3

Allocation:
P0: 0 1 0
P1: 2 0 0
P2: 3 0 2
P3: 2 1 1

Need:
P0: 7 4 3
P1: 1 2 2
P2: 6 0 0
P3: 0 1 1

[SUCCESS] Sistem dalam SAFE STATE.
Safe sequence: P1 P2 P3 P0

=== REQUEST dari P1 ===
Meminta: 1 0 2

[SUCCESS] Sistem dalam SAFE STATE.
Safe sequence: P1 P2 P3 P0
[DITERIMA] Request aman, dialokasikan.

=== STATUS SISTEM ===
Available: 4 4 1

Allocation:
P0: 0 1 0
P1: 3 0 2
P2: 3 0 2
P3: 2 1 1

Need:
P0: 7 4 3
P1: 0 2 0
P2: 6 0 0
P3: 0 1 1

=== REQUEST dari P3 ===
Meminta: 3 3 0
[DITOLAK] Request melebihi need!

=== PROGRAM SELESAI ===
dea@LAPTOP-H54VQ76S:~/Praktikum_Deadlock$ |
```

Cara kerja program:

- Need dihitung otomatis → Max – Allocation □ User memasukkan:
 - Nomor proses (0–3)

- Resource yang ingin di-request (3 angka) □ Program:
- Mengecek apakah request valid
- Melakukan alokasi sementara
- Mengecek apakah sistem tetap aman
- Jika aman maka request disetujui
- Jika tidak maka rollback dan request ditolak

TUGAS 4 KASUS NYATA (REAL CASE)

Buat simulasi deadlock untuk kasus: "Dua orang ingin transfer uang antar rekening secara bersamaan"

- Orang A: transfer dari Rekening 1 ke Rekening 2
- Orang B: transfer dari Rekening 2 ke Rekening 1
- Implementasikan deadlock prevention

Buat file transfer_deadlock.c

```

GNU nano 7.2 transfer_deadlock.c
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t rekening1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t rekening2 = PTHREAD_MUTEX_INITIALIZER;

void* transfer_A(void* arg) {
    printf("A: Mengunci Rekening 1...\n");
    pthread_mutex_lock(&rekening1);
    sleep(1);

    printf("A: Mencoba mengunci Rekening 2...\n");
    pthread_mutex_lock(&rekening2);

    printf("A: Transfer selesai\n");

    pthread_mutex_unlock(&rekening2);
    pthread_mutex_unlock(&rekening1);
    return NULL;
}

void* transfer_B(void* arg) {
    printf("B: Mengunci Rekening 2...\n");
    pthread_mutex_lock(&rekening2);
    sleep(1);

    printf("B: Mencoba mengunci Rekening 1...\n");
    pthread_mutex_lock(&rekening1);

    printf("B: Transfer selesai\n");

    pthread_mutex_unlock(&rekening1);
    pthread_mutex_unlock(&rekening2);
    return NULL;
}

int main() {
    pthread_t A, B;

    printf("=== SIMULASI DEADLOCK TRANSFER UANG ===\n");

    pthread_create(&A, NULL, transfer_A, NULL);
    pthread_create(&B, NULL, transfer_B, NULL);

    pthread_join(A, NULL);
    pthread_join(B, NULL);

    return 0;
}

```


Penjelasan:

- `pthread_mutex_t rekening1, rekening2` ini untuk 2 mutex mewakili 2 rekening yang harus di lock sebelum transfer.
- `pthread_mutex_lock(&rekening1) (A)`
Ini untuk A mengunci rekening 1 terlebih dahulu.
- `pthread_mutex_lock(&rekening2) (A)` ini untuk A mencoba mengunci rekening 2, tetapi akan gagal karena B sudah memegangnya.
- `pthread_mutex_lock(&rekening2) (B)`
ini adalah B mengunci rekening 2 terlebih dahulu.
- `pthread_mutex_lock(&rekening1) (B)` ini untuk B mencoba mengunci rekening 1, tetapi gagal karena A sudah memegangnya.
- Hasil A menunggu B dan B menunggu A (Deadlock).

Outputnya:

```
dea@LAPTOP-H54VQ765:~/Praktikum_Deadlock$ nano transfer_deadlock.c
dea@LAPTOP-H54VQ765:~/Praktikum_Deadlock$ gcc transfer_deadlock.c -o dlock -lpthread
./dlock
=== SIMULASI DEADLOCK TRANSFER UANG ===
A: Mengunci Rekening 1...
B: Mengunci Rekening 2...
A: Mencoba mengunci Rekening 2...
B: Mencoba mengunci Rekening 1...
|
```

Cara Kerja Program:

- A mengunci Rekening 1
- B mengunci Rekening 2
- A butuh Rek. 2 yang dikunci B
- B butuh Rek. 1 yang dikunci A

Jadi yang terjadi adalah kondisi terus saling menunggu dan terjadilah deadlock.

Buat file transfer_prevention_deadlock.c

```
GNU nano 7.2 transfer_prevent.c
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t rekening1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t rekening2 = PTHREAD_MUTEX_INITIALIZER;

// fungsi helper untuk lock berurutan
void lock_ordered() {
    pthread_mutex_lock(&rekening1);
    pthread_mutex_lock(&rekening2);
}

void unlock_ordered() {
    pthread_mutex_unlock(&rekening2);
    pthread_mutex_unlock(&rekening1);
}

void* transfer_A(void* arg) {
    printf("A: Mulai transfer (1 → 2)\n");
    lock_ordered();
    printf("A: Berhasil mengunci kedua rekening\n");
    sleep(1);
    unlock_ordered();
    printf("A: Transfer selesai\n");
    return NULL;
}

void* transfer_B(void* arg) {
    printf("B: Mulai transfer (2 → 1)\n");
    lock_ordered();
    printf("B: Berhasil mengunci kedua rekening\n");
    sleep(1);
    unlock_ordered();
    printf("B: Transfer selesai\n");
    return NULL;
}

int main() {
    pthread_t A, B;
    printf("=== PENCEGAHAN DEADLOCK: TRANSFER UANG ===\n");
    pthread_create(&A, NULL, transfer_A, NULL);
    pthread_create(&B, NULL, transfer_B, NULL);
    pthread_join(A, NULL);
    pthread_join(B, NULL);
    printf("\nProgram selesai tanpa deadlock.\n");
    return 0;
}
```

Penjelasan:

- lock_ordered(from, to) ini adalah fungsi menentukan urutan penguncian berdasarkan ID rekening.
- first = from < to ? from : to ini untuk menentukan rekening mana yang lebih kecil ID nya.
- pthread_mutex_lock(&rekening[first]) ini agar selalu mengunci rekening nomor kecil lebih dahulu.
- pthread_mutex_lock(&rekening[second]) ini agar mengunci nomor besar.
- unlock_ordered() ini untuk melepas lock dengan urutan terbalik.
- Hasil yang terjadi adalah thread mengunci rekening dengan urutan yang sama dan deadlock dicegah.

Outputnya:

```
dea@LAPTOP-H54VQ765:~/Praktikum_Deadlock$ nano transfer_prevent.c
dea@LAPTOP-H54VQ765:~/Praktikum_Deadlock$ gcc transfer_prevent.c -o prevent -lpthread
./prevent
=== PENCEGAHAN DEADLOCK: TRANSFER UANG ===
A: Mulai transfer (1 → 2)
A: Berhasil mengunci kedua rekening
B: Mulai transfer (2 → 1)
A: Transfer selesai
B: Berhasil mengunci kedua rekening
B: Transfer selesai

Program selesai tanpa deadlock.
dea@LAPTOP-H54VQ765:~/Praktikum_Deadlock$ |
```

Cara kerja program:

- Setiap rekening harus dikunci sebelum dipakai, agar tidak dipakai 2 orang sekaligus
- Program selalu mengunci rekening dengan urutan yang sama, rekening bernomor kecil dulu, kemudian rekening bernomor besar
- Ketika A dan B mentransfer bersamaan:
A : Rekening 1 ke 2
B : Rekening 2 ke 1
Jadi keduanya akan tetap mengikuti aturan yang sama, yaitu kunci rekening 1 dulu lalu rekening 2
- Akibatnya hanya 1 orang yang bisa mendapatkan 2 kunci yang lainnya otomatis menunggu sebentar, tidak selamanya dan kemudian deadlock tidak akan terjadi.